```
[2013-08-07 02:00:01] INFO  WEBrick::HTTPServer#start: pid=69680
port=3000
```

With just three commands we whipped up a Rails server listening on port 3000. Go to your browser and open http://localhost:3000, you will see a basic Rails app running.
You can also use the alias "s" to start the server: `rails s`.

The server can be run on a different port using the `-p` option. The default development environment can be changed using `-e`.

```
$ bin/rails server -e production -p 4000
```

The `-b` option binds Rails to the specified IP, by default it is localhost. You can run a server as a daemon by passing a `-d` option.

1.3 `rails generate`

The `rails generate` command uses templates to create a whole lot of things. Running `rails generate` by itself gives a list of available generators:
You can also use the alias "g" to invoke the generator command: `rails g`.

```
$ bin/rails generate
Usage: rails generate GENERATOR [args] [options]

...
...

Please choose a generator below.

Rails:
  assets
  controller
  generator
  ...
  ...
```

You can install more generators through generator gems, portions of plugins you'll undoubtedly install, and you can even create your own!

Using generators will save you a large amount of time by writing **boilerplate code**, code that is necessary for the app to work.
Let's make our own controller with the controller generator. But what command should we use? Let's ask the generator:

All Rails console utilities have help text. As with most *nix utilities, you can try adding `--help` or `-h` to the end, for example `rails server --help`.

```
$ bin/rails generate controller
Usage: rails generate controller NAME [action action] [options]

...
...

Description:
    ...

    To create a controller within a module, specify the controller
name as a path like 'parent_module/controller_name'.

    ...

Example:
    `rails generate controller CreditCards open debit credit close`
```